rview Preparation Guide

BDD • Gherkin • Step Definitions • Hooks • Ta

BDD Framework | Gherkin Syntax | Step Definitions | Hooks & Tags | Selenium + Cucumber | TDD vs BDD

Section 1 — Cucumber & BDD Basics

Q1. What is Cucumber?

Cucumber is an open-source testing tool used for **Behavior-Driven Development (BDD)**. It lets you write test cases in plain English so that **everyone on the team** — developers, testers, and business people — can read and understand them without needing technical knowledge.

Think of it as a bridge between the business world and the technical world. Instead of writing tests only in code, you describe what the software should do in simple sentences, and Cucumber automatically runs those sentences as tests.

Originally written in Ruby, Cucumber now supports Java, JavaScript, Python, .NET, and many more.

Q2. What is Behavior-Driven Development (BDD)?

BDD is a development approach where the whole team — business analysts, developers, and testers — **collaborates to define how the software should behave** before any code is written.

The main idea: describe the software's behavior in simple, human-readable language (using examples), so everyone has the same understanding. This prevents miscommunication and ensures the software truly meets business needs.

- Uses the 'Five Whys' technique to trace every feature back to a real business outcome.
- Combines the best of TDD (Test-Driven Development) and ATDD (Acceptance Test-Driven Development).
- Tests serve as living documentation that everyone can read.

Q3. What is Gherkin Language?

Gherkin is the **plain English language** that Cucumber uses to write test cases. It is designed to be readable by non-technical people.

Gherkin uses special **keywords** to structure the tests:

- **Feature** – What big functionality are we testing?
- **Scenario** – One specific test case (a single situation).
- **Given** – The starting state / precondition.
- **When** – The action the user or system performs.
- **Then** – The expected result we want to see.
- **And / But** – Used to add more steps without repeating Given/When/Then.
- **Background** – Steps that run before every scenario in the file.
- **Examples** – A data table used with Scenario Outline.

Gherkin supports 37+ spoken languages, so you can write tests in languages other than English too.

Q4. What is a Feature in Cucumber?

A **Feature** is a high-level description of one piece of functionality in your software. It is stored in a file called a **Feature File** (extension: **.feature**).

Example: For an e-commerce website, features could be: User Login, Add to Cart, Checkout, Payment, etc. Each feature gets its own .feature file.

A feature file contains: the feature name, a short description, and one or more scenarios that test different situations of that feature.

Best practice: Keep one feature per file and a maximum of ~10 scenarios per file for readability.

Q5. What is a Scenario in Cucumber?

A **Scenario** is a single, specific test case inside a feature file. It describes one particular situation — what happens when a user does something specific.

Each scenario has a sequence of steps using Given, When, Then keywords. Example:

```
Scenario: Successful user login
  Given the user is on the login page
  When the user enters valid username and password
  Then the user should be redirected to the homepage
```

Each scenario is independent — it should not depend on another scenario to run correctly.

Q6. What is a Scenario Outline?

A **Scenario Outline** is used when you want to run the **same test scenario multiple times with different data**. Instead of copy-pasting the same steps, you write a template with placeholders and provide data in an Examples table.

Example:

```
Scenario Outline: Login with different credentials
  Given the user is on the login page
  When the user enters "" and ""
  Then the result should be ""

  Examples:
    | username | password | message       |
    | admin    | admin123 | Welcome admin |
    | guest    | guest123 | Welcome guest |
```

Cucumber will run this scenario twice — once for each row in the Examples table.

Q7. What are the two files required to run a Cucumber test?

You need exactly two types of files:

- **Feature File (.feature)** — Written in Gherkin. Describes what the software should do (the test scenario in plain English).
- **Step Definition File** — Written in a programming language (Java, Ruby, etc.). Contains the actual code that runs when each Gherkin step is executed.

The feature file says 'WHAT to test', and the step definition file says 'HOW to test it'.

Q8. What is a Step Definition?

A **step definition** is a method/function in your code that is linked to a Gherkin step. When Cucumber reads a step like *'Given the user is on the login page'*, it looks for a matching step definition and runs that code.

Example in Java:

```
@Given("the user is on the login page")
public void userOnLoginPage() {
    driver.get("https://example.com/login");
}
```

Step definitions use annotations like @Given, @When, @Then to tell Cucumber which step they match.

Section 2 — Gherkin Keywords Explained

Q9. What does the 'Given' keyword mean?

Given sets up the **initial context or starting state** before any action happens. Think of it as answering the question: 'What is the situation before we begin?'

Example: *Given the user has $100 in their bank account* — this tells us the starting state before a withdrawal is attempted.

Given usually refers to something that already happened before the scenario begins.

Q10. What does the 'When' keyword mean?

When describes the **action or event** — what the user does or what happens in the system. It is the core of the test — the thing you are actually testing.

Example: *When the user clicks the 'Withdraw' button with amount $50*

Q11. What does the 'Then' keyword mean?

Then describes the **expected outcome** — what should happen as a result of the action. This is where you verify that the system behaved correctly.

Example: *Then the account balance should show $50*

Then is where assertions/validations happen in the step definition code.

Q12. What do 'And' and 'But' keywords do?

And lets you add multiple steps to a Given, When, or Then without repeating the keyword. It improves readability.

```
Given the user is logged in
And the user has items in the cart
When the user clicks checkout
And the user selects express shipping
Then the order should be confirmed
But the free shipping option should not be available
```

But works exactly like And but is used to express a negative or contrasting condition, making the scenario more natural to read.

Q13.

What is the Background keyword used for?

The **Background** keyword lets you define steps that should run **before every scenario** in a feature file. It avoids repetition when every scenario needs the same setup.

```
Feature: Shopping Cart

Background:
  Given the user is logged in
  And the user is on the product page

Scenario: Add item to cart
  When the user clicks Add to Cart
  Then the cart should show 1 item

Scenario: Remove item from cart
  When the user removes an item
  Then the cart should be empty
```

Both scenarios above will first run the Background steps automatically.

Q14.

What is the difference between Background and Scenario Outline?

Background: Runs a fixed set of common steps before EVERY scenario in the file. Used for shared setup (like logging in).

Scenario Outline: Runs the SAME scenario multiple times with DIFFERENT data sets from an Examples table. Used for data-driven testing.

Simple rule: Use Background to avoid repeating setup steps. Use Scenario Outline to avoid repeating similar scenarios.

Section 3 — Annotations, Hooks & Tags

Q15. What are Annotations in Cucumber?

Annotations are **special markers** placed on methods in your step definition files. They tell Cucumber what type of step the method handles.

- **@Given** — Marks a method that sets up preconditions.
- **@When** — Marks a method that performs an action.
- **@Then** — Marks a method that checks the outcome.
- **@Before** — Marks a method that runs before each scenario (setup).
- **@After** — Marks a method that runs after each scenario (cleanup).

The text in the annotation (e.g., @Given("the user is on the login page")) is used to match the Gherkin step.

Q16. What are Hooks in Cucumber?

Hooks are **blocks of code that run automatically at specific points** during the test — before or after each scenario. They handle setup and teardown tasks.

Why we need them: Without hooks, you'd have to repeat the setup and cleanup code in every single scenario. Hooks centralize this logic.

```
@Before
public void setUp() {
    // Open browser, set up test data, etc.
    driver = new ChromeDriver();
}

@After
public void tearDown() {
    // Close browser, clear test data, etc.
    driver.quit();
}
```

Common uses: opening/closing browser, setting up database connections, taking screenshots on failure, clearing cookies.

Q17. What is the difference between @Before (Hook) and Background?

Both run before each scenario, but they are different:

- **Background** is written inside a .feature file in Gherkin. It is visible to non-technical people. It only applies to scenarios in that one feature file.
- **@Before Hook** is written in code (step definition files). It can apply across ALL feature files. You can use tag expressions to run it only for specific scenarios.

Use Background for business-readable setup steps. Use @Before hooks for technical setup like browser initialization.

Q18. What are Tags in Cucumber? Why are they important?

Tags are **labels** you put on scenarios or features using the @ symbol. They help you organize and selectively run tests.

```
@SmokeTest
Scenario: Verify user login
  Given the user is on the login page
  When the user enters valid credentials
  Then the user should be logged in
```

Why tags are important:

- **Filter tests** — Run only @SmokeTest scenarios: *cucumber --tags @SmokeTest*
- **Exclude tests** — Skip tests tagged @Skip: *cucumber --tags 'not @Skip'*
- **Organize tests** — Group tests as @Regression, @API, @UI, @WIP, etc.
- **Environment control** — Run different sets of tests in different environments.

Q19. How do you skip a scenario in Cucumber?

Tag the scenario with a custom tag (like @Skip) and then exclude that tag when running the tests:

```
@Skip
Scenario: This scenario will be skipped
  Given some setup
  When some action
  Then some result
```

Run command: **cucumber --tags 'not @Skip'**

This way, skipped scenarios are clearly visible in the feature file but won't execute during the test run.

Section 4 — Key Cucumber Concepts

Q20.

What is a Data Table in Cucumber?

A **Data Table** lets you pass **multiple rows of data** to a single step. It is used when you want to test something with many values at once (like submitting a form with multiple fields, or validating a list).

```
Scenario: Verify user registration details
  Given the following users are registered:
    | name  | email            | age |
    | Alice | alice@test.com   | 28  |
    | Bob   | bob@test.com     | 35  |
```

In the step definition, you access this table using a DataTable object and iterate through rows.

Difference from Scenario Outline: Data Tables pass multiple values to ONE step. Scenario Outline runs the WHOLE scenario multiple times.

Q21.

What is a Cucumber Profile?

A Cucumber **Profile** is a named group of settings that tells Cucumber how to run — which tags to include, where to find feature files, which format to output, etc.

Profiles are defined in a **cucumber.yml** file:

```
default:  --tags @Regression --plugin pretty
smoke:    --tags @Smoke --plugin html:reports/smoke.html
dev:      --tags @WIP
```

To run a specific profile: **cucumber --profile smoke**

This is useful when you want different test configurations for different environments or purposes without changing your code.

Q22. What is a Dry Run in Cucumber?

A **Dry Run** checks that all steps in your feature files have matching step definitions — **without actually running the tests**. It is a quick way to catch missing or unimplemented steps.

```
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    dryRun = true
)
```

If a step has no matching step definition, Cucumber shows a warning and suggests the code you need to write.

Use dry run when: you add new steps to feature files and want to check nothing is missing before running the full suite.

Q23. What is a Cucumber Report? How do you generate an HTML report?

A **Cucumber Report** is a document showing which tests passed, failed, or were skipped — with details about each step, execution time, and error messages if any.

To generate an HTML report, configure the plugin in @CucumberOptions:

```
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    plugin = {"pretty", "html:target/cucumber-report.html"}
)
```

After running, open **target/cucumber-report.html** in a browser to see a visual, color-coded test report.

Other formats: **json** (for CI/CD tools), **junit** (for Jenkins/pipeline integration), **pretty** (colored console output).

Q24. What are Cucumber Parameters?

Cucumber **Parameters** allow you to pass dynamic values from your Gherkin step into the step definition method. This makes your steps reusable across different data values.

```
# Feature file step:
Given the user orders 3 pizzas

# Step definition:
@Given("the user orders {int} pizzas")
public void userOrdersPizzas(int quantity) {
    cart.add("pizza", quantity);
}
```

Common parameter types:

- **{int}** — captures an integer number
- **{string}** — captures a quoted string
- **{float}** — captures a decimal number
- **{word}** — captures a single word

Q25. What is the role of Regular Expressions in Cucumber?

Regular expressions (regex) are **patterns used in step definitions** to match Gherkin steps. They allow one step definition to match multiple variations of a step and extract values from it.

```
# Matches: "the user enters 'John' as the username"
@Given("the user enters \"([^\"]*)\" as the username")
public void enterUsername(String username) {
    loginPage.enterUsername(username);
}
```

The pattern **([^"]*)** captures any text inside quotes and passes it as the 'username' parameter. This is how dynamic values flow from your feature file into your code.

Q26.

What is the Cucumber World?

The **World** is a **shared context object** in Cucumber that holds data and state which needs to be accessed by multiple steps within the same scenario.

Instead of using global variables (which can cause problems), you store data in the World object so it is isolated per scenario.

```
public class ScenarioContext {
    public String username;
    public String authToken;
}

// In step definition:
@Given("the user logs in as {string}")
public void loginAs(String username) {
    scenarioContext.username = username;
}
```

In Ruby-based Cucumber, World is a special built-in object. In Java, dependency injection (like PicoContainer) achieves the same effect.

Section 5 — Cucumber with Selenium & Other Tools

Q27. How do you use Cucumber with Selenium?

Cucumber provides the test scenarios (the 'what'), and **Selenium WebDriver** provides the browser automation (the 'how'). Together they form a complete web testing framework.

Setup steps:

- **Step 1:** Add Cucumber + Selenium dependencies to pom.xml (Maven).
- **Step 2:** Write feature files describing user interactions.
- **Step 3:** Write step definitions that use Selenium to automate the browser.
- **Step 4:** Create a TestRunner class with @CucumberOptions.
- **Step 5:** Run with *mvn test*.

Example step definition using Selenium:

```
@When("the user enters valid username and password")
public void enterCredentials() {
    driver.findElement(By.id("username")).sendKeys("testUser");
    driver.findElement(By.id("password")).sendKeys("pass123");
    driver.findElement(By.id("loginBtn")).click();
}
```

Q28. Why use Cucumber with Selenium together?

Each tool has a different strength:

- **Selenium** — automates browser actions (click, type, navigate). But test scripts are pure code — hard for non-technical people to read.
- **Cucumber** — adds a human-readable layer on top. Business analysts and managers can read and even write scenarios.

Together: Test scenarios are readable by the whole team AND automated by Selenium. This improves collaboration, reduces miscommunication, and makes tests serve as documentation.

Q29. What software is needed to run Cucumber web tests?

- **JDK/JRE** — Java development kit (if using Java).
- **IDE** — IntelliJ IDEA or Eclipse to write code.
- **Maven/Gradle** — Build tool to manage dependencies.
- **Cucumber libraries** — cucumber-java, cucumber-junit.
- **Selenium WebDriver** — For browser automation.
- **Browser Driver** — ChromeDriver, GeckoDriver etc. matching your browser version.

Q30.

How do you run Cucumber tests in parallel?

Running tests in parallel means multiple tests run at the same time, which reduces the total execution time significantly.

In Java with Cucumber-JVM, you can use the **Cucumber JVM Parallel Plugin**. It scans your feature files and creates separate test runners for each.

Key requirement: Tests must be **independent** — they should not share any mutable state. Use @Before and @After hooks to ensure each test starts clean.

In Ruby, the parallel_tests gem is commonly used for this purpose.

Q31.

How do you integrate Cucumber into a CI/CD pipeline?

CI/CD integration means your Cucumber tests run automatically every time code is pushed.

- **Step 1:** Add a test execution step in your CI config (Jenkins, GitLab CI, GitHub Actions).
- **Step 2:** Run tests with: *mvn test* (Maven) or *bundle exec cucumber* (Ruby).
- **Step 3:** Configure Cucumber to output JUnit XML or JSON reports (CI tools can read these).
- **Step 4:** Set the pipeline to FAIL if any Cucumber tests fail — this blocks bad code from reaching production.
- **Step 5:** Use tags to run different test sets for different stages (e.g., @Smoke for quick checks, @Regression for full run).

Section 6 — TDD vs BDD, JUnit vs Cucumber, RSpec vs Cucumber

Q32. **What is Test-Driven Development (TDD)?**

TDD is a development approach where you **write tests BEFORE writing the actual code**. The cycle is:

- **1. Write a failing test** — for a new piece of functionality.
- **2. Write minimal code** — just enough to make the test pass.
- **3. Run tests** — confirm they pass.
- **4. Refactor** — clean up the code while keeping tests green.
- **5. Repeat** — for the next piece of functionality.

Result: Higher code quality, better test coverage (90-100%), and more maintainable code.

Q33. **What are the similarities between TDD and BDD?**

- Both write tests BEFORE writing production code (test-first approach).
- Both promote automated testing throughout development.
- Both encourage frequent testing to catch bugs early.
- Both include code refactoring as a step.
- Both aim to produce higher quality, reliable code.

Q34. **What are the key differences between TDD and BDD?**

Here is a clear comparison:

- **Focus:** TDD focuses on testing individual code units. BDD focuses on the behavior of the whole system from the USER's perspective.
- **Language:** TDD tests are written in code (JUnit, NUnit). BDD uses plain English (Gherkin in Cucumber).
- **Audience:** TDD is mainly for developers. BDD involves developers, testers, AND business people.
- **Scope:** TDD tests small, isolated pieces of code. BDD tests end-to-end features and user workflows.
- **Documentation:** TDD produces technical code tests. BDD produces readable documentation for everyone.

Q35.

What is the difference between JUnit and Cucumber?

JUnit is a unit testing framework for Java. You write tests in pure Java code to test individual methods or classes in isolation. Mainly used by developers.

Cucumber is a BDD framework. Tests are written in Gherkin (plain English) and test the whole system's behavior from the user's perspective. Used by the whole team.

Important: In a Cucumber + Java project, **JUnit is used as the test runner** to execute Cucumber scenarios. They work together — they are not mutually exclusive.

Q36.

What is the difference between RSpec and Cucumber?

RSpec is a testing framework for Ruby used for unit and BDD testing within code. Tests are written in a Ruby DSL (Domain Specific Language). Best for testing individual Ruby classes/methods.

Cucumber is a BDD acceptance testing tool. Tests are written in plain Gherkin English. Can be understood by non-technical stakeholders. Tests the full system behavior.

Key distinction: RSpec keeps specs inside code — closer to the developer. Cucumber separates specs from code — making them accessible to the whole team.

Q37.

What is the difference between JBehave and Cucumber?

Both JBehave and Cucumber are BDD frameworks, but they differ:

- **Language:** JBehave is pure Java. Cucumber started in Ruby and now supports many languages including Java.
- **Terminology:** JBehave uses 'Stories'. Cucumber uses 'Features'.
- **Setup:** JBehave requires more configuration. Cucumber is generally easier to set up.
- **Community:** Cucumber has a larger community, more plugins, and better third-party tool integration.
- **Use case:** JBehave suits Java-centric teams. Cucumber is more flexible and widely adopted.

Section 7 — Advanced Cucumber Topics

Q38. How do you handle asynchronous operations in Cucumber tests?

Async operations (like waiting for a page to load, an API to respond, or an animation to finish) can cause tests to fail if not handled properly.

Best approaches:

- **Explicit waits** — Wait for a specific condition (element visible, page loaded) up to a timeout: *WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));*
- **Polling** — Repeatedly check for a condition at intervals instead of waiting a fixed time.
- **Async/Await** — In JavaScript Cucumber, use async/await to handle Promise-based operations cleanly.

Avoid Thread.sleep() or plain time delays — they make tests slow and brittle.

Q39. How do you share state between steps in the same scenario?

Steps in a scenario often need to share data (e.g., a user ID created in a Given step needs to be used in a Then step). Options:

- **Instance variables** in the step definition class — simple but can cause coupling.
- **Dependency Injection** (PicoContainer, Spring) — Cucumber creates and injects a shared context object into all step definition classes. This is the recommended approach.

```
public class ScenarioContext {
    public String orderId;
}
// Injected into step definitions via constructor:
public OrderSteps(ScenarioContext ctx) { this.ctx = ctx; }
```

Avoid global/static variables — they leak state between scenarios and cause unpredictable failures.

Q40. What is soft assertion vs hard assertion?

Hard Assertion: Stops the test immediately when it fails. The remaining steps/assertions are NOT executed.

Soft Assertion: Continues executing even after a failure. At the end, all failures are reported together.

```
// Hard assertion (stops immediately):
Assert.assertEquals(expected, actual);

// Soft assertion (collects all failures):
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals(val1, expected1);
softAssert.assertEquals(val2, expected2);
softAssert.assertAll(); // Reports all failures
```

Use hard assertions for critical checks where subsequent steps make no sense if the check fails. Use soft assertions when you want to validate multiple independent conditions and report all failures at once.

Q41. How do you handle flaky (intermittently failing) tests in Cucumber?

Flaky tests pass sometimes and fail sometimes — they are unreliable. Common causes and fixes:

- **Timing issues** → Use explicit waits, not fixed sleep delays.
- **Shared/leaked state** → Clean up data in @After hooks, use unique test data per scenario.
- **External service instability** → Mock the external service in tests.
- **Race conditions (parallel tests)** → Ensure test isolation; use separate databases or transactions.
- **Retry mechanism** → Use a retry plugin to automatically re-run failed scenarios 1-2 times.

Q42. How do you test APIs using Cucumber?

API testing with Cucumber involves using Cucumber for the test structure (Gherkin scenarios) and an HTTP client library for making actual API calls.

Common libraries: **RestAssured** (Java), **requests** (Python), **rest-client** (Ruby).

```
Scenario: Get user details via API
  Given a user with ID 42 exists
  When I send a GET request to "/api/users/42"
  Then the response status code should be 200
  And the response body should contain "email"
```

API tests are faster and more stable than UI tests because they bypass the browser and test the backend directly.

Q43. What strategies help manage a large Cucumber test suite?

- **Folder structure** — Organize feature files by module/feature in separate directories.
- **Tags** — Use @Smoke, @Regression, @API, @WIP etc. to categorize and selectively run tests.
- **Limit scenarios per file** — Max 10 scenarios per feature file for readability.
- **Reusable step definitions** — Extract common steps into shared step definition files.
- **Regular cleanup** — Periodically remove outdated or redundant scenarios.
- **Parallel execution** — Run independent scenarios simultaneously to reduce total run time.

Q44. How do you test different user roles/permissions with Cucumber?

Create scenarios that explicitly set up different user roles in the Given step:

```
Scenario: Admin can access admin dashboard
  Given I am logged in as an Admin
  When I navigate to the admin panel
  Then I should see the admin dashboard

Scenario: Regular user cannot access admin panel
  Given I am logged in as a Regular User
  When I navigate to the admin panel
  Then I should see an 'Access Denied' message
```

The step definition for *'Given I am logged in as {string}'* handles the logic of logging in with the appropriate credentials for each role.

Q45. How do you debug a failing Cucumber test?

- **Read the error message carefully** — Cucumber shows which step failed and why.
- **Add print/log statements** — Print variable values inside step definitions to trace what's happening.
- **Set breakpoints in IDE** — Run in debug mode to pause and inspect state at the failing step.
- **Check test data** — Ensure the data in your feature file is correct and matches what the system expects.
- **Dry run first** — Confirm all steps have matching step definitions.
- **Isolate the test** — Run only the failing scenario to eliminate interference from other tests.
- **Check environment** — Verify the test environment is set up correctly (database, browser, network).

Q46.

What is grouping in Cucumber?

Grouping means **organizing related scenarios and step definitions** together so the test suite is manageable. Two main approaches:

1. Using Tags — Tag scenarios with @Smoke, @Regression, @Login etc.

2. Folder organization — Place feature files in directories by module:

```
features/
  login/        → Login.feature
  registration/ → Registration.feature
  cart/         → Cart.feature
```

Also group step definitions into packages matching the feature areas for cleaner, navigable code.

Q47.

What is the maximum recommended number of scenarios per feature file?

Best practice: Keep a **maximum of 10 scenarios per feature file**.

Why? Feature files with too many scenarios become hard to read, navigate, and maintain. If a file grows beyond 10 scenarios, consider splitting it into sub-features.

There is no hard technical limit — Cucumber can run files with 100 scenarios. But 10 is the recommended number for readability and maintainability.

Section 8 — Languages, Plugins & Quick Reference

Q48. Which programming languages does Cucumber support?

Cucumber supports a wide range of programming languages:

- **Ruby** — Original implementation (cucumber gem).
- **Java** — Cucumber-JVM, most widely used in enterprise.
- **JavaScript** — Cucumber.js (Node.js).
- **.NET / C#** — SpecFlow (Cucumber port for .NET).
- **Python** — Behave (similar to Cucumber).
- **PHP** — Behat.
- **Groovy, Kotlin, Scala** — via Cucumber-JVM.

Gherkin itself supports 37+ spoken languages, so feature files can be written in languages other than English.

Q49. What are Cucumber Plugins?

Cucumber plugins **extend or enhance Cucumber's behavior** — especially for reporting and integration. You configure them in @CucumberOptions:

```
plugin = {"pretty", "html:target/report.html", "json:target/report.json"}
```

- **pretty** — Colored, readable console output.
- **html:path** — Generates an HTML report at the given path.
- **json:path** — Generates a JSON report (useful for CI/CD).
- **junit:path** — Generates JUnit XML report (compatible with Jenkins).

External plugins like **cucumber-reporting** or **Extent Reports** provide even more detailed HTML reports with charts and history.

Q50.

What is a Test Harness in Cucumber?

A test harness is the **complete set of tools, libraries, and infrastructure** that work together to enable automated Cucumber test execution. It includes:

- Feature files (Gherkin test scenarios)
- Step definition files (code implementations)
- Test runner class (connects features to step definitions)
- Build tool (Maven/Gradle) managing dependencies
- Hooks and plugins for setup, teardown, and reporting
- Browser/API automation tools (Selenium, RestAssured)

The test harness separates concerns — your step definitions focus on interaction logic, while the harness handles execution orchestration.

Q51.

What is the difference between a Feature, Scenario, and Step?

Think of it as a hierarchy:

- **Feature** (biggest) — One whole functionality (e.g., User Authentication).
- **Scenario** (inside a Feature) — One specific test case (e.g., Successful Login).
- **Step** (inside a Scenario) — One single action or check (e.g., *Given the user is on the login page*).

One Feature file → has multiple Scenarios → each Scenario has multiple Steps.

Q52.

How do you pass data from a Feature file to a Step Definition?

There are three main ways:

- **Inline arguments** — Pass values directly in the step text using {int}, {string} etc.
- **Data Tables** — Pass a structured table of multiple values to one step.
- **Scenario Outline Examples** — Pass different values to run the same scenario multiple times.

```
# Inline argument example:
Given the user orders {int} items
# Step definition captures 'int' as a parameter

# Data table example:
Given the following products:
  | name   | price |
  | Apple  | 1.00  |
  | Banana | 0.50  |
```

Q53.

What are best practices for writing Cucumber step definitions?

- **Keep steps atomic** — One step should do one thing only.
- **Use parameters** — Make steps reusable by capturing dynamic values with {int}, {string} etc.
- **DRY principle** — Don't repeat step code. Extract common logic to helper methods.
- **Descriptive names** — Step text should read like natural language.
- **No Given/When/Then inside step definitions** — Keep them as simple orchestrators.
- **Group by feature** — Organize step definition files by feature/module.
- **Avoid UI locators in feature files** — Use Page Object Model in step definitions to abstract UI details.

Q54.

What is the Page Object Model (POM) and how does it relate to Cucumber?

Page Object Model is a design pattern where each page of your web application is represented by a class. That class contains all the locators and methods for interacting with that page.

In Cucumber + Selenium: your step definitions call Page Object methods instead of directly using Selenium commands. This means:

- If the UI changes, you only update the Page Object class — not every step definition.
- Step definitions stay clean and readable.
- Code is highly reusable across scenarios.

POM makes Cucumber tests resilient to UI changes — a critical practice for large test suites.

Q55.

When should you NOT use Cucumber?

Cucumber is great for acceptance/BDD testing, but it's not always the right tool:

- **Unit testing** — Use JUnit, pytest, NUnit, or RSpec instead. They're faster and simpler for testing individual methods in isolation.
- **Performance testing** — Use JMeter, Gatling, or k6. Cucumber is not designed for load simulation.
- **When no non-technical stakeholders are involved** — The overhead of Gherkin may not be worth it for purely developer-internal projects.
- **Very simple applications** — A small script or API with 2-3 endpoints may not need BDD overhead.

Follow the testing pyramid: many unit tests, fewer integration tests, fewer still end-to-end Cucumber tests.

Q56.

How do you convince a team to adopt Cucumber/BDD?

Present it as a collaboration and communication tool, not just a testing tool:

- Show how Gherkin scenarios replace long requirement documents — everyone can read and verify them.
- Run a small pilot on one feature. Show the readable feature file to a business stakeholder — let the results speak.
- Demonstrate that catching a misunderstood requirement early (in BDD planning) is 10x cheaper than fixing it after development.
- Emphasize that tests become living documentation — always up to date.

Start small. A successful pilot on one well-defined feature is more persuasive than any argument.

Q57.

How do you keep Cucumber tests updated when requirements change?

Requirements change constantly — here's how to keep tests current:

- Participate in sprint planning/backlog grooming to learn about upcoming changes early.
- Review and update feature files as part of every story/ticket — not as an afterthought.
- Use version control (Git) for feature files so changes are tracked.
- Delete or archive scenarios that no longer reflect real behavior — stale tests cause confusion.
- Run Cucumber in CI/CD — failing tests after a code change immediately signal that feature files need updating.

Q58.

How do you manage external dependencies like databases in Cucumber tests?

External dependencies (databases, queues, APIs) can make tests slow and unreliable if not managed properly:

- **Separate test database** — Never test against the production database. Use a dedicated test DB.
- **Reset state in @Before** — Clean/reset the database before each scenario for isolation.
- **Use transactions** — Roll back database changes after each test so state doesn't leak.
- **Mock external APIs** — Use WireMock or similar to simulate external services.
- **Docker** — Use Docker Compose to spin up fresh, consistent database containers for tests.

Q59. What types of reports can Cucumber generate?

- **HTML Report** — Visual, color-coded report viewable in a browser. Best for stakeholders.
- **JSON Report** — Machine-readable format. Used by CI/CD tools and custom dashboards.
- **JUnit XML** — Compatible with Jenkins and most CI tools for test result display.
- **Pretty (console)** — Human-readable colored output in the terminal during test runs.
- **External tools** — cucumber-reporting, Allure Reports, Extent Reports for advanced dashboards with charts and history.

Q60. What is the @CucumberOptions annotation and what are its key parameters?

@CucumberOptions is the configuration annotation on your TestRunner class. It tells Cucumber where to find things and how to run:

```
@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",  // Where feature files are
    glue = "com.example.stepdefs",             // Where step definitions are
    tags = "@Regression",                       // Which tags to run
    dryRun = false,                             // true = validate only, no run
    plugin = {"pretty",                         // Output format
              "html:target/report.html"},
    monochrome = true                           // Clean console output
)
public class TestRunner { }
```

- **features** — Path to .feature files.
- **glue** — Package containing step definitions.
- **tags** — Filter which scenarios to run.
- **plugin** — Output/report formatters.
- **dryRun** — Validate steps without running.

Quick Reference — Cucumber Cheat Sheet

Concept	What it is	Key Point
Feature	High-level description of a functionality	.feature file, one per feature area
Scenario	A single test case	Has Given/When/Then steps
Scenario Outline	Template to run same scenario with multiple data	Uses Examples table
Background	Common setup steps before each scenario	Defined once, runs before every scenario
Data Table	Structured multi-row data passed to one step	Different from Scenario Outline
Step Definition	Code that runs for each Gherkin step	Linked via @Given/@When/@Then
Hook @Before	Code that runs before each scenario	Setup: browser, data, DB connection
Hook @After	Code that runs after each scenario	Cleanup: close browser, reset data
Tag	Label (@Smoke, @Regression) on scenarios	Use to filter which tests to run
Dry Run	Validates step definitions exist	dryRun = true, no actual test execution
Gherkin	The language used to write feature files	Given / When / Then / And / But
World	Shared state object within a scenario	Avoids global variables
Profile	Named config group in cucumber.yml	Different settings for different test runs
Plugin	Adds reporting/integration capability	html, json, junit, pretty

■ Good luck with your Cucumber interview!

Cover all sections, understand the concepts clearly, and practice writing feature files and step definitions by hand.